

# DB Whisperer

מערכת אינטראקטיבית לחקר מסדי נתונים בשפה טבעית

פרויקט מסכם - סמינר הנדסת תוכנה בעידן ה-AI

טלי ראטנר

לירן בכר

אוהד ליואי

יותם כץ

|    |                                |
|----|--------------------------------|
| 3  | 1. שאלת המחקר, מטרות וחשיבותן  |
| 3  | 1.1. הבעיה ההנדסית             |
| 3  | 1.2. שאלת המחקר                |
| 3  | 1.3. יעדי הפרויקט              |
| 3  | 1.4. חשיבות ותרומה צפויה       |
| 3  | 2. הרקע ההנדסי והשיטה המוצעת   |
| 3  | 2.1. רקע: NL-to-SQL ועמימות    |
| 4  | 2.2. ארכיטקטורת המערכת         |
| 5  | 2.3. כלים וטכנולוגיות          |
| 5  | 3. מימוש הפתרון והערכתו        |
| 5  | 3.1. קלטים ועיבוד מקדים        |
| 5  | 3.2. רכיבים מרכזיים במימוש     |
| 6  | 3.3. תכנון הערכת המערכת        |
| 6  | 3.4. תוצאות ההערכה             |
| 7  | 3.5. דיון וממצאים              |
| 7  | 4. תהליך הפיתוח, אתגרים ולקחים |
| 7  | 4.1. שלבי העבודה               |
| 7  | 4.2. אתגרים והחלטות טכנוניות   |
| 8  | 4.3. לקחים מקצועיים והנדסיים   |
| 8  | 5. סיכום ועבודה עתידית         |
| 8  | 5.1. מסקנות מרכזיות            |
| 8  | 5.2. יתרונות                   |
| 9  | 5.3. חסרונות ומגבלות           |
| 9  | 5.4. כיוונים לעבודה עתידית     |
| 9  | 6. מדריך למשתמש                |
| 9  | 6.1. דרישות מערכת ותלויות      |
| 9  | 6.2. שימוש במערכת באתר         |
| 9  | 6.3. שימוש והתקנה מקומית       |
| 9  | 6.4. שלבי פעולת המערכת         |
| 10 | 6.5. קלטים, פלטים ושגיאות      |
| 10 | 6.6. מבנה המאגר                |

## 1. שאלת המחקר, מטרות וחשיבותן

### 1.1. הבעיה ההנדסית

מערכות NL-to-SQL אמורות לתרגם שאלה בשפה טבעית לשאילתת מסד נתונים. האתגר איננו רק לכתוב SQL תקין תחבירית, אלא להבין למה המשתמש התכוון כאשר הניסוח חלקי או רב-משמעי, לבחור את הטבלאות והעמודות הנכונות, לבצע Joins מדויקים, ולמנוע פעולה לא בטוחה. במאגר רב-טבלאי אותה מילה יכולה להתאים למספר שדות: למשל, 'שנה' יכולה להתייחס לשנת לידה, לשנת קבלה לבית חולים או לשנת מדידה. תשובה שנראית סבירה עלולה להיות נכונה מבחינה תחבירית אך שגויה מבחינת הכוונה. למשתמש שאינו טכני קשה לזהות טעות כזו, מפני שהוא בדרך כלל אינו מכיר את הסכמה ואינו קורא SQL ולעיתים אינו מודע בכלל לאילו מידעים קיימים בכלל בנתונים, ואילו לא. לכן הבעיה ההנדסית היא לבנות תהליך מקצה לקצה שמנגיש את הנתונים, אך גם מציג אי-ודאות בצורה שימושית, שומר על בטיחות, ומוודא שהחלטת המשתמש אכן מיושמת בתוצאה הסופית.

### 1.2. שאלת המחקר

באיזו מידה מערכת מבוססת LLM להמרת שפה טבעית ל-SQL יכולה לסייע למשתמשים לא-טכניים לחקור מסדי נתונים מרובי טבלאות ועמודות, תוך צמצום פרשנות שגויה באמצעות זיהוי מפורש של עמימות ושאלות הבהרה?

### 1.3. יעדי הפרויקט

- לאפשר העלאה או בחירה של קבצי CSV ושאילת שאלות אודותם בשפה טבעית.
- גלולת טבלאות, עמודות וקשרים אפשריים, ולהשתמש בהקשר הסכמה ליצירת SQL.
- להפיק שאילתת SELECT יחידה, תקינה וקריאה בלבד, ולהציג למשתמש את התוצאה ואת השאילתה.
- לזהות עמימות בשפת המשתמש, להציג זאת למשתמש על מנת לשפר את דיוק והתאמת השאילתה.
- להעריך באופן מבוקר את תרומת מנגנוני העמימות

### 1.4. חשיבות ותרומה צפויה

אנחנו מאמינים שלפרויקט שלנו יש חשיבות רבה בהנגשת נתונים למשתמשים בעלי ידע מקצועי, אך ללא ידע ב-SQL. מערכת כזאת מעצימה משתמשי קצה מגוונים ובעלת השלכות אמיתיות בעולמות העסקיים ואף במתן ביטוי לעקרון חופש המידע. ממשיק בשפה טבעית יכול לקצר את זמן החיפוש ולאפשר חקירה עצמאית. עם זאת, הנגשה ללא מנגנון שמטפל באי-ודאות עלולה ליצור ביטחון שגוי. התרומה הצפויה של DB Whisperer היא שילוב בין נגישות, שקיפות ובקרה.

## 2. הרקע ההנדסי והשיטה המוצעת

### 2.1. רקע: NL-to-SQL ועמימות

במערכת NL-to-SQL, מודל שפה מקבל את שאלת המשתמש ואת תיאור הסכמה ומפיק שאלת SQL שניתן להריץ. איכות התוצאה תלויה גם בבחירת ההקשר הנכון: שמות הטבלאות והעמודות, סוגי הנתונים, דוגמאות לערכים והקשרים בין הטבלאות. ככל שהסכמה גדולה יותר, הצגת כולה למודל נעשית יקרה ומבלבלת. לכן נדרש Schema Linking: בחירת הטבלאות והשדות הרלוונטיות לשאלת המשתמש.

אתגר מרכזי נוסף הוא עמימות בשאלת המשתמש. עמימות נוצרת כאשר לשאלה יש יותר מפרשנות טבעית אחת הנתמכת בסכמה (למשל: "כמה מטופלים סבלו משפעת?" - לא ברור חתך הזמן, איזה סוג שפעת, חולים כעת או אי פעם חלו?). עם זאת, לא כל הבדל בין שאלות SQL מעיד על עמימות אמיתית. ההבדל יכול לנבוע מניסוח שונה של אותה שאלתה, מבחירה מקרית של המודל או משאלתה שאינה מתאימה לכוונת המשתמש.

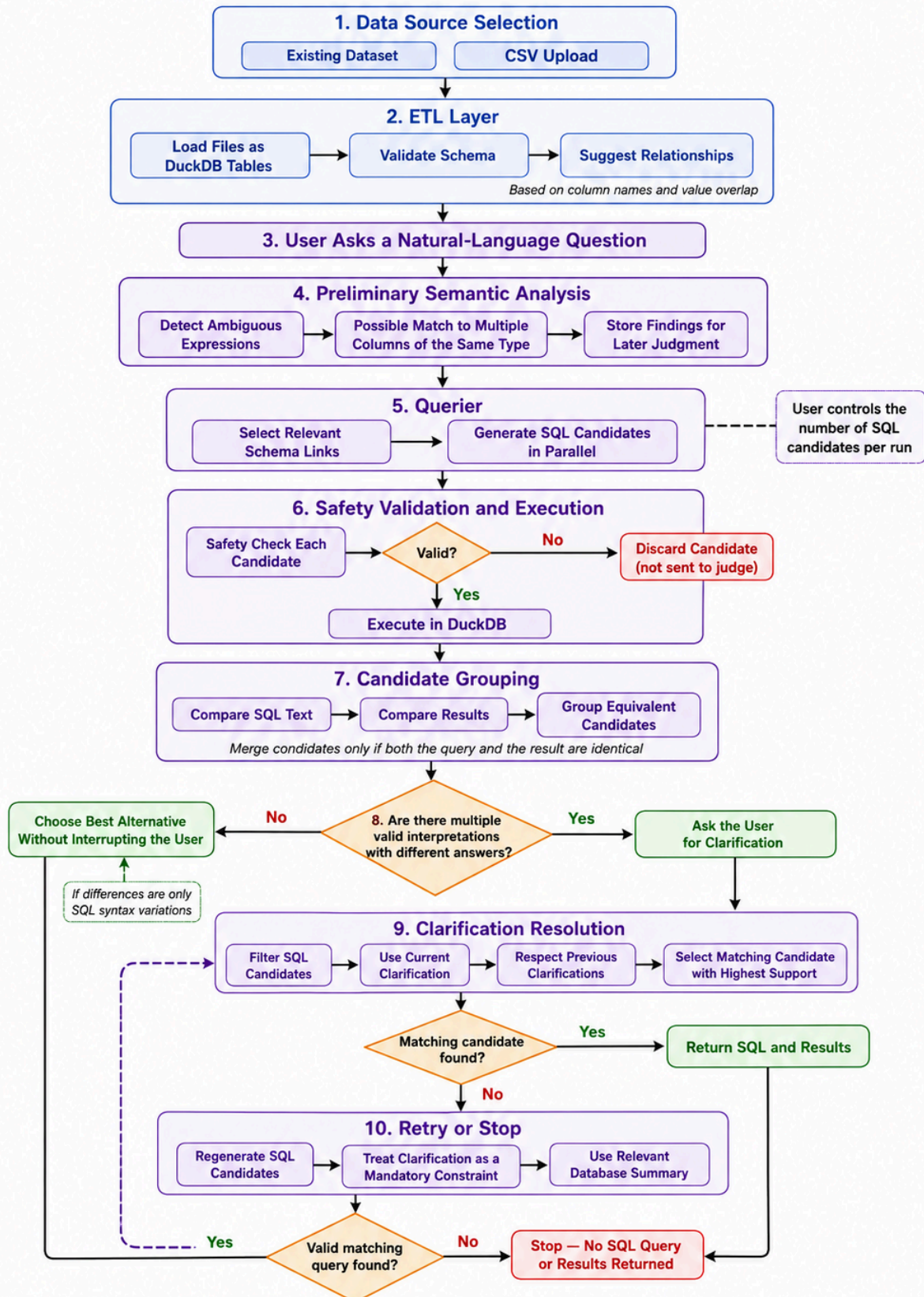
השיטה המוצעת ב-DB Whisperer היא גישה אינטראקטיבית ל-NL-to-SQL, שמטרתה לשפר את אמינות התשובות באמצעות זיהוי עמימות והבהרת כוונת המשתמש בעת הצורך. במקום להסתמך על שאלתה יחידה, השיטה בוחנת כמה פרשנויות אפשריות של השאלה ומשלבת מידע מהסכמה, מהמשמעות הלשונית של השאלה ומהתוצאות שהתקבלו בפועל. כאשר מזוהות שתי פרשנויות סבירות ושונות באופן מהותי, המערכת מבקשת מהמשתמש לבחור ביניהן. כאשר אין עמימות משמעותית, היא מחזירה את החלופה המתאימה ביותר ללא שאלה נוספת. כך השיטה מנסה לאזן בין דיוק ואמינות לבין שמירה על תהליך שימוש פשוט שאינו מעמיס על המשתמש.

### 2.2. ארכיטקטורת המערכת

| רכיב                           | אחריות                                                                                                                 |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Component A - ETL              | טעינת CSV ל-DuckDB, חילוץ מטא-דאטה וגילוי קשרים אפשריים.                                                               |
| Component B - Hybrid Ambiguity | ניתוח סמנטי מקדים, השוואת שאלות SQL שנוצרו והחלטה אם לשאול הבהרה.                                                      |
| Component C - Querier          | בניית prompt, יצירת SQL, בדיקת בטיחות והרצה.                                                                           |
| Component D - GUI              | ממשק משתמש הכולל אפשרות בחירת מערך נתונים, העלאת קובצי CSV, הזנת שאלות, הצגת אפשרויות הבהרה והצגת שאלת ה-SQL והתוצאות. |

# System Flow

Natural-language querying with SQL candidate generation, validation, clarification, and retry logic



| כלי        | שימוש בפרויקט                                                             |
|------------|---------------------------------------------------------------------------|
| Python     | שפת המימוש של כל רכיבי המערכת וההערכה.                                    |
| Streamlit  | ממשק אינטרנטי, הצגת הצ'אט, הסכמה, תוצאות, SQL והבהרות.                    |
| DuckDB     | טעינת CSV והרצת שאילתות אנליטיות מקומית.                                  |
| OpenRouter | גישה נוחה וגמישה למגוון רחב של מודלי שפה, הן בצד המתכנת והן בצד המשתמש.   |
| SQLGlott   | ניתוח ובדיקת SQL, כולל אכיפת שאילתת קריאה יחידה.                          |
| unittest   | בדיקות אוטומטיות לרכיבי ETL, Querier, Ambiguity, Application, GUI והערכה. |

### 3. מימוש הפתרון והערכתו

#### 3.1. קלטים ועיבוד מקדים

הקלטים הידניים הם קובץ CSV אחד או יותר עם שורת כותרות, שאלת משתמש, בחירת מודל ומפתח API כאשר הוא נדרש. המשתמש יכול גם לבחור מערך הדגמה מובנה. לאחר שאלת הבהרה נוסף לקלט גם הפירוש שבחר המשתמש. בעיבוד האוטומטי, כל CSV נטען לטבלה נפרדת. המערכת מחלצת שמות וטיפוסים מדויקים, מספרי שורות, דוגמאות וסטטיסטיקות בסיסיות. היא מציעה קשרים בין טבלאות באמצעות שילוב של רמזים סמנטיים בשם העמודה וחפיפת ערכים. לאחר מכן נבחר הקשר הסכמה המתאים לשאלה. בסכמות גדולות, schema linking שומר את הטבלאות הרלוונטיות ומוסיף שרשרת קשר קצרה ודטרמיניסטית כאשר נדרשות טבלאות גישור.

#### 3.2. רכיבים מרכזיים במימוש

- **app.py**: משמש כנקודת הכניסה של המערכת המפעילה את streamlit.
- **etler**: אחראי לטעינת הקבצים וזיהוי הקשרים בין טבלאות ושדות.
- **querier**: בניית הפרומפטים, שאילתות SQL, בדיקתן והרצתן.
- **ambiguity**: לניתוח כוונה עמומה ולשיפוט חלופות.
- **gui**: מציג את הUI של המערכת, קליטת הקלטים והצגת הפלטים.
- **contracts.py**: משמש כחוזת למודלים המשותפים בין הרכיבים.

מימוש ההבהרה כולל מזהים יציבים לחלופות ולממצאים סמנטיים. השופט חייב להחזיר בדיוק שתי אפשרויות תקפות, ולאחר הבחירה עליו לסווג כל חלופה מול התשובות שנקבעו. כאשר ההבהרה מצביעה על עמודה בטבלה שלא נבחרה מהשאלה המקורית, הטבלה ננעצת בהקשר הסכמה לאחר הבחירה, כדי לאפשר ליוצר ה-SQL להשתמש בה. הנעיצה אינה מתבצעת לפני הבחירה, וכך נמנעת הטיה של קבוצת המועמדים הראשונה.

#### 3.3. תכנון הערכת המערכת

גרסת ההערכה הנוכחית של המערכת (V3), היא אבולוציה של 2 גרסאות קודמות (V1, V2). מערך קבוע של 22 שאלות מכסה תשובות ישירות, עמימות סמנטית, עמימות הנגזרת ממועמדים, פתרון

הבהרות, בטיחות ובקשות יחסיות בין טבלאות. שני fixtures נוספים בדקו את שכבת ה-ETL. כל זרוע קיבלה את אותן שאלות ואת אותו מצב מסד נתונים.

| זרוע           | K | מטרה                            | מידע לעמימות                                     |
|----------------|---|---------------------------------|--------------------------------------------------|
| Baseline       | 1 | קו בסיס של יצירת SQL ישירה.     | ללא שכבת עמימות                                  |
| Candidate Only | 3 | לבודד את תרומת השוואת המועמדים. | הבדלים בין SQL ותוצאות שבוצעו                    |
| Semantic Only  | 3 | לבודד את תרומת ניתוח העמודות.   | ממצאים סמנטיים וסכמה; חלופות לאחר תשובה רק לציות |
| Full System    | 3 | לבדוק את הצינור ההיברידי המלא.  | מועמדים + ממצאים סמנטיים + הקשר סכמה             |

(\*) מס' השאלות המיוצרות כמועמדים  $k = \text{מס'}$

כל 22 השאלות הורצו בארבע הזרועות בחמש חזרות:  $22 = 5 * 4 * 5 = 440$  תצפיות שאילתה. שני תרחישי ETL משותפים הורצו בכל חזרה:  $2 * 5 = 10$  תצפיות נוספות. סך הכול: 450 תצפיות תקינות.

הניקוד הדטרמיניסטי בחן את התוצאה מול תוצאה מוסכמת ואת עדויות ה-SQL, התייחס לנכונות סמנטית ולחיבורים המינימליים המספיקים, ובחן את משפך העמימות ואת אמינות ובטיחות המערכת: האם הייתה עמימות, האם היה מוצדק לעצור, איכות השאלה, והאם הבחירה יושמה בתוצאה.

### 3.4 תוצאות ההערכה

| זרוע           | ציון משוקלל | שיעור מעבר (Pass/Fail) | נכונות סמנטית |
|----------------|-------------|------------------------|---------------|
| Baseline       | 44.53       | 47.62%                 | 67.06%        |
| Candidate Only | 45.62       | 47.62%                 | 70.59%        |
| Semantic Only  | 67.43       | 60.95%                 | 81.18%        |
| Full System    | 76.54       | 69.52%                 | 83.53%        |

#### תוצאה מרכזית

Full System עלתה על Baseline ב-32.01 נקודות בציון המשוקלל וב-21.90 נקודות אחוז בשיעור המעבר. שאר הזרועות שיפרו את ביצועי המערכת ביחס ל-baseline כל אחת בפני עצמה אך השילוב שלהן לתוך המערכת המלאה הביא את התוצאה המירבית.

### 3.5 דיון וממצאים

השוואת המועמדים לבדה שיפרה מעט את הנכונות הסמנטית, אך לא שינתה את שיעור המעבר ביחס לקו הבסיס. לעומת זאת, הזרוע הסמנטית השיגה שיפור גדול יותר, והמערכת המלאה השיגה את התוצאה הטובה ביותר בכל שלושת המדדים. דפוס זה מצביע על כך ששונות בין מועמדי SQL אינה מספיקה ללא ראייה שמחברת את ההבדל לניסוח ולסכמה. השילוב ההיברידי מאפשר להשתמש בתוצאות שבוצעו כראיה חזקה ובניתוח העמודות כרשת ביטחון וכהקשר. עם זאת, יש לפרש את התוצאה בזהירות. מדובר במערך סטטי, במודל ובקובץ מסד נתונים מסוימים, ולא בניסוי משתמשים או בהוכחה כללית לכל תחום. אופי הניקוד בגרסה הסופית שינה כללים שהענישו עמודות הקשר מיותרות, aliases לא מזיקים, ייצוגי משך שקולים וסדרי tie-break

שלא נדרשו מהמשתמש. מקרה אחד שבו משמעות 'frequency' נשארה בלתי פתורה הוצא מהמכנה המרכזי, אך כל הראיות נשמרו והוצגה גם בדיקת רגישות.

#### 4. תהליך הפיתוח, אתגרים ולקחים

##### 4.1. שלבי העבודה

**תשתית מקצה לקצה:** תכנון של ארבעת הרכיבים של המערכת, פיתוח ומימוש של כל אחד מהם בנפרד תוך שמירה על חוזי תקשורת ברורים.

**מעבר לנתונים יחסיים:** הרחבה מקובץ יחיד למספר טבלאות, גילוי קשרים, הקשר סכמה מחובר ותצוגת מטא-דאטה.

**שיפור לוגיקת העמימות:** מעבר מריבוי מסלולי חיבור כהוכחה לעמימות להחלטה מאוחדת המבוססת על חלופות שבוצעו וממצאים סמנטיים.

**ראיות והערכה:** בניית ארבע זרועות מבודדות, חזרות, checkpoints, תיעוד פר-מקרה וניקוד דטרמיניסטי שניתן לביקורת ולשחזור.

##### 4.2. אתגרים והחלטות תכנוניות

- 1. ריבוי מסלולי חיבור אינו בהכרח ריבוי כוונות:** בגרסה קודמת ריבוי מסלולים בגרף הסכמה הפעיל שאלת הבהרה. בפועל, מסלולים שונים יכולים להחזיר אותה משמעות או לעבור דרך טבלת עובדות באופן שאינו טבעי. לכן הוסר המנגנון כהוכחת עמימות. schema linking עדיין משתמש בקשר קצר כדי לתמוך ביצירת SQL, אך אינו מסיק שעצם קיום מסלולים חלופיים מחייב שאלה.
- 2. שונות בין מועמדים מול outlier של מודל:** חלופה יחידה יכולה להיות פירוש אמיתי, אך גם טעות מקרית. נבחרה בדיקת סבירות שמעריכה אם שתי האפשרויות טבעיות, קוהרנטיות והדדית מובחנות. מספר התמיכה משמש כרשת ביטחון, כדי לא למחוק פירוש מיעוט לגיטימי.
- 3. מידת הדטרמיניסטיות של מודל:** במקרה של טעות, לעיתים לא ניתן להבחין האם הטעות עקבית ומערכתית או שהמודל פשוט טעה בהטלה הספציפית הזאת. לכן בחרנו לתת למודלים רמת דטרמיניסטיות נמוכה (Temperature), זאת כדי לצמצם את מרחב האפשרויות כך שהתשובות בעלות שונות פחותה, וטעויות מיוחסות לבעיה במערכת, למשל בפרומפט מסוים, מאשר במודל.
- 4. מניעת הטיה ביצירת השאלות:** שימוש מוקדם בממצאי הניתוח הסמנטי עלול לכוון את כל מועמדי ה-SQL לאותה פרשנות. לכן המועמדים הראשונים נוצרים ללא הכוונה זו, והממצאים משמשים רק לאחר ההרצה לבדיקת העמימות מתוך מועמדי ה-SQL. לאחר תשובת המשתמש, המערכת משתמשת במידע הסמנטי כדי להתאים אותו לסכמה ולייצר שאלתה שתואמת לבחירתו.
- 5. תשובת הבהרה שלא השפיעה על ה-SQL:** המערכת זיהתה נכון שקיימת עמימות ושאלה את המשתמש למה התכוון, אבל שאלת ה-SQL שהוחזרה לאחר מכן לא תמיד תאמה לבחירתו. במקרים כאלה, העדפנו שהמערכת תדווח שלא הצליחה למצוא שאלתה מתאימה, מאשר שתחזיר תשובה שאינה מכבדת את בחירת המשתמש.
- 6. ניקוד מחמיר מדי בהערכת המערכת:** כללים מוקדמים דחו תשובות שקולות בגלל aliases, עמודות הקשר, סדר עמודות או תיקו בתוצאות. הצוות בדק ידנית את הראיות ברמת המקרה והפריד בין נכונות הכוונה לבין דיוק ההצגה והיעילות. התוצאות המקוריות נשמרו, והניקוד המתוקן פורסם כארטיפקט נפרד.
- 7. אמינות ריצה, עלות ושחזור:** קריאות מודל הן משתנות ועוללות להיכשל מסיבות רבות. מסגרת V3 השתמשה בתקציב קבוע, fingerprints של סביבת הבדיקה והקוד,



checkpoints אטומיים, יכולת resume, הפרדה בין כשל מערכת לכשל תשתית, ושער אישור לפני פרסום הדוחות. בנוסף, נבחר שלא להשתמש ב-LLM כבחון של התוצאות (במקום הבדיקה הידנית), בעיקר משיקולי עלות.

#### 4.3. לקחים מקצועיים והנדסיים

- **הבדלים בין מודלים:** השתמשנו ב-OpenRouter כדי לאפשר למשתמש הקצה את החופש להשתמש בכל מודל, גם חדשים שיצאו בעתיד. אך בניסויים שלנו גילינו שהמודלים מאוד מגוונים ביכולות שלהם לעמוד במשימות והוראות, בשימוש בכלים, בקונטקסט ארוך ובעיות מורכבות (Jagged Intelligence - Karpathy) - כך שביצועי המערכת תלויים מאוד מהבחירה של המודל, ללא אפשרות נוחה לדעת מראש איזה מודל עשוי להתאים מבלי לבצע הערכה יקרה. נוצר כאן tradeoff בין חופש משתמש לבחור decoupling ממודל ספציפי, לבין רמת אמינות וביצועי המערכת כולה.
- **שאלות שונות, תוצאה זהה:** גילינו כי שיפוט העמימות מושפע מדי ממבנה השאלות, ולא מהתוצאות שלהן. כלומר ייתכנו שתי שאלות שכתובות בצורה שונה, אך בפועל מחזירות את אותה תוצאה, ושופט העמימות יחליט שיש צורך בשאלת הבהרה.
- שאלת הבהרה אינה שלב תצוגה בלבד; היא חייבת להפוך לאילוץ שנבדק מול התוצאה הסופית, לשמש כראייה נוספת בעת בניית השאלתה.
- מערכת מורכבת הנבנית על ידי מפתחים שונים במקביל מרוויחה מאוד מהפרדה מודולרית בין הרכיבים שלה (ETL, יצירת שאלות, שיפוט, UI). כך שכל מפתח יכול להתמקצע בתחום שלו. בנוסף - טעויות אינן נגררות, הן מטופלות ברמת component בשל ה-decoupling הטבעי שנוצר בעקבות ההפרדה.
- קשה מאוד לאבחן ולשפר מערכת מורכבת ומרובת שלבים. לעיתים שיפור אחד גורר ריגרסיה במקומות אחרים. לכן יש לייצר ממשק הערכה end-to-end כדי לשמר ולאמוד שיפור בביצועים, יש לאסוף מסודר לוגים ויצירת ממשק Observability שנותן למפתח להבין לעומק את ה-flow והיכן אירעה הטעות או התקלה.
- כאשר לא ניתן להוכיח התאמה לכוונת המשתמש, עדיף כישלון ברור על פני תשובה נוחה אך מטעה.
- הבחירה לזהות עמימות על ידי יצירת מועמדי SQL והשוואתם היא לא הבחירה הטבעית שהיינו מבצעים - והיא מביאה עמה יתרונות אך גם חסרונות משמעותיים.
- ניתן להוזיל עלויות בעזרת שינוי מבנה הפרומפט: קודם החלקים הסטטיים שניתנים ל-caching, ואז החלקים הדינמיים. הוזלה של עד 80% מהעלויות במקרים מסויימים.

#### 5. **סיכום ועבודה עתידית**

##### 5.1. מסקנות מרכזיות

DB Whisperer ממחישה שאפשר לבנות צינור NL-to-SQL נגיש ושקוף עבור נתוני CSV רב-טבלאיים, תוך שילוב בין גילוי סכמה, יצירת שאלות SQL בטוחות, הצגת תוצאה והבהרה ממוקדת. במסגרת ההערכה V3, טיפול מפורש בעמימות שיפר את המדדים לעומת יצירה ישירה. הממצא החשוב איננו שהמערכת שאלה יותר שאלות, אלא שהיא ניסתה לשאול שאלה מדויקת רק כאשר קיימות שתי פרשנויות סבירות, ולאחר מכן בדקה שהבחירה יושמה.

##### 5.2. יכולות ויתרונות

- גישה למערכי נתונים בשפה טבעית למשתמשים שאינם יודעים SQL.
- תמיכה במספר קובצי CSV ובגילוי קשרים בין טבלאות.

- שאלת הבהרה ממוקדת במקרה של אי-בהירות.
- שקיפות באמצעות הצגת SQL, תוצאות, סכמה והיסטוריית שיחה.
- ארכיטקטורה מודולרית וביצוע שמרני לקריאה בלבד.
- הערכה מבוקרת עם ארבע זרועות, חזרות וראיות לכל הרצה.

### 5.3. חסרונות ומגבלות

- מספר קריאות API מגבירה את זמן התגובה (במידה רבה במקרים של קריאות סדרתיות ולא מקבילות), העלויות והשונות בין הרצות.
- הקשרים שה-ETL מזהה אוטומטית הם קשרים אפשריים המבוססים על הנתונים, ולא קשרים ודאיים שהוגדרו במסד הנתונים.
- איכות SQL, ניתוח סמנטי ושאלת הבהרה תלויה במודל שנבחר.
- גרסת ההערכה מבוססת על mimic-iii אחד ואינה מחליפה ניסוי עם משתמשים לא-טכניים.
- מקרים שבהם אף מועמד SQL אינו מכבד את ההבהרה עדיין מסתיימים בכישלון. אם גם לאחר ניסיון נוסף לא נוצרת שאילתה שתואמת לתשובת המשתמש, המערכת עוצרת ואינה מחזירה תוצאה.
- לוגים עשויים להכיל מידע מהקבצים שהועלו ולכן דורשים מדיניות פרטיות ושמירה ברורה.

### 5.4. כיוונים לעבודה עתידית

- הרחבת המערכת לתמיכה בסוגי דטא שונים (לא רק רלציוני, לא רק CSV), שימוש ב-LLM כבחון בבדיקת המערכת.
- מחקר משתמשים עם קהל לא-טכני, כולל אמון, שימושיות וזיהוי טעויות נפוצות.
- שיפור התאוששות כאשר אין מועמד תואם לאחר הבהרה, בלי להחזיר תשובה לא מאומתת.
- שיפור זיהוי כוונה סמנטית ודירוג הקשר הסכמה בסכמות גדולות ורועשות.
- בקורות פרטיות ברורות ללוגים, דוגמאות נתונים ושמירת שיחות.
- entity resolution ושיפור הסינון הסמנטי של קשרים שהתגלו אוטומטית.

## 6. מדריך למשתמש

### 6.1. דרישות מערכת ותלויות

- לשימוש בגרסה המתארכת: דפדפן מודרני וחיבור לאינטרנט.
- לשימוש מקומי: Python, Git והחבילות המוגדרות ב-requirements.txt.
- מפתח OpenRouter API כאשר המודל המובנה בגרסה המתארכת אינו זמין.
- קובץ CSV אחד או יותר עם שורת כותרות, או שימוש במערך טבלאות מובנה.

### 6.2. שימוש במערכת באתר

1. פתחו את היישום בקישור המופיע במסמך.
2. בחרו מערך נתונים מובנה או העלו קובצי CSV קשורים.
3. בדקו וחקרו את הטבלאות, העמודות, הדוגמאות והקשרים שהמערכת זיהתה.
4. בחרו מודל OpenRouter והזינו מפתח API אם נדרש.
5. כתבו שאלה על הנתונים בשפה טבעית.
6. אם מוצגת שאלה עם שתי פרשנויות, בחרו בזו שמתאימה לכוונתכם.
7. בדקו את טבלת התוצאה ואת ה-SQL שהפיק אותה.

### שימוש והתקנה מקומית 6.3.

מתיקיית השורש של המאגר, התקינו את התלויות והפעילו את Streamlit:

```
python -m pip install -r requirements.txt
streamlit run app.py
```

### שלבי פעולת המערכת 6.4.

1. טעינת קובצי ה-CSV ל-DuckDB ובדיקת הסכמה.
2. גילוי קשרים אפשריים ובחירת הקשר סכמה רלוונטי.
3. יצירה, אימות והרצה של מועמדי SQL.
4. שיפוט אם ההבדלים מייצגים עמימות מהותית.
5. החזרת תוצאה מאומתת או שאלה אחת עם שתי אפשרויות.
6. לאחר בחירה: בדיקת תאימות, ניסיון תיקון אחד, או כישלון גלוי.

### קלטים, פלטים ושגיאות 6.5.

| סוג       | פירוט                                                                    |
|-----------|--------------------------------------------------------------------------|
| קלטים     | טבלאות CSV, שאלה בשפה טבעית, בחירת מודל, מפתח API ותשובת הבהרה אם נשאלה. |
| פלט הצלחה | טבלת תוצאות, SQL שנוצר, הקשר רלוונטי והיסטוריית השיחה.                   |
| פלט הבהרה | שאלה ממוקדת אחת ושתי אפשרויות ברורות.                                    |
| פלט כשל   | הודעה ברורה כאשר טעינה, יצירה, אימות, הרצה או ציות להבהרה נכשלו.         |

### מבנה המאגר 6.6.

|                   |                                        |
|-------------------|----------------------------------------|
| app.py            | Streamlit entry point                  |
| src/db_whisperer/ |                                        |
| application/      | Workflow orchestration                 |
| etl/              | CSV ingestion and relationships        |
| querier/          | Prompting, SQL validation, execution   |
| ambiguity/        | Intent and candidate evaluation        |
| gui/              | Streamlit presentation                 |
| contracts.py      | Shared data contracts                  |
| benchmark_v3/     | Current controlled evaluation          |
| /tests            | Automated test suite                   |
| /docs             | Evaluation and design records          |
| /submission       | Final website and submission materials |

## מאמרים 7.

### CodexGraph: Bridging Large Language Models and Code Repositories via Code 7.1.

Graph Databases. Liu, X., Lan, B., Hu, Z., et al. (2024). *arXiv*.

<https://doi.org/10.48550/arxiv.2408.03910>

#### 7.1.1 תקציר המאמר: מטרת המאמר היא לשפר את היכולת של מודלי שפה לבצע משימות

הדורשות הבנה של מאגר קוד שלם, ולא רק של קטע קוד בודד. הגישה המוצעת, CODEXGRAPH, ממירה את מאגר הקוד למסד נתונים גרפי המתאר רכיבים כמו מחלקות, פונקציות והקשרים ביניהם, ומאפשרת למודל לאתר מידע באמצעות שאלות

גרף. התרומה העיקרית היא יצירת ממשק אחיד שאינו מוגבל למשימת תכנות מסוימת ויכול לשמש לניווט, אחזור מידע והבנת מבנה הקוד. ממצאי המחקר מצביעים על כך ששימוש במסד נתונים גרפי מסייע למודלי שפה להתמודד עם מגוון משימות ברמת מאגר הקוד ולהשיג ביצועים תחרותיים מול שיטות אחזור קיימות. לצד זאת, הגמישות של השיטה כרוכה בצריכת טוקנים גבוהה יותר.

**7.1.2. תרומת המאמר:** המאמר חיזק את החלטתנו לספק למודל מידע מסודר על מבנה מסד הנתונים ועל הקשרים שהתגלו בין הטבלאות, במקום להתייחס לטבלאות שהועלו כאל טקסט לא מובנה. בהתאם לעיקרון זה, הפרדנו בין בחירת המידע הרלוונטי מהסכמה לבין יצירת שאלת ה-SQL הסופית על ידי המודל.

**7.2. ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification.**  
Mu, F., Shi, L., Wang, S., et al. (2023). *arXiv*.  
<https://doi.org/10.48550/arxiv.2310.10996>

**7.2.1. תקציר המאמר:** מטרת המאמר היא לשפר את תהליך יצירת הקוד כאשר דרישות המשתמש אינן ברורות או מלאות. ClarifyGPT מייצרת כמה פתרונות ובודקת אם הם מתנהגים באופן דומה. כאשר מתקבלות תוצאות שונות, המערכת מזהה עמימות אפשרית, מציגה למשתמש שאלות הבהרה ממוקדות ומעדכנת את הדרישה לפי תשובותיו לפני יצירת הקוד הסופי. התרומה המרכזית של המאמר היא שילוב מנגנון אינטראקטיבי שמאפשר למודל לזהות חוסר בהירות ולברר את כוונת המשתמש במקום להניח למה התכוון. הממצאים מצביעים על כך שהשיטה משפרת את דיוק הקוד במגוון מודלים ומערכי נתונים, הן בעזרת תשובות של משתמשים אמיתיים והן בעזרת תשובות שנוצרו באופן מלאכותי.

**7.2.2. תרומת המאמר:** המאמר סיפק את הבסיס לרעיון המרכזי של הפרויקט: יצירת כמה פתרונות אפשריים, שימוש בהבדלים ביניהם לזיהוי עמימות ושאלת המשתמש רק כאשר נדרשת הבהרה. DB Whisperer התאימה את הגישה לתחום מסדי הנתונים באמצעות השוואה בין שאלות SQL ובין התוצאות שהתקבלו לאחר הרצתן, לצד שימוש במידע סמנטי מסכמת הנתונים. במהלך הפיתוח בחרנו להרחיב את הרעיון כך שהבדל בין מועמדים נחשב רק לסימן אפשרי לעמימות, ושאלת הבהרה מוצגת רק כאשר קיימות שתי פרשנויות שונות משמעותית.